Aprendizaje supervisado y no supervisado

Clase 8 · Función de pérdida · PCA · k-means

Daniela Opitz
INF-396 · Departamento de Informática · UTFSM · viernes 30 de octubre, 2026

Avisos

- **Hoy, al inicio de la clase, el Control Q3: 20 minutos, sobre los informes de OpenAI y del AISI.**
- La Tarea 2 está publicada en el repositorio, con su plantilla y sus datos. Se entrega el lunes 16 de noviembre por Aula, en equipos de máximo dos personas: hay que inscribir el equipo en el Aula (Unidad 6), aunque sea el mismo de la Tarea 1.
- El avance de proyecto es opcional y se presenta el viernes 6 de noviembre: quienes lo presenten parten con 10 puntos de base.

Contenidos de la clase

- 1. Tipos de aprendizaje: supervisado y no supervisado.
- 2. La función de pérdida en el aprendizaje supervisado.
- 3. Reducción de dimensionalidad: análisis de componentes principales (PCA).
- 4. Clustering con k-means.

Bloque 1 · Aprender de los datos

Tipos de aprendizaje, función de pérdida y análisis de componentes principales

1. Tipos de aprendizaje

Supervisado, no supervisado y por refuerzo

Paradigmas de aprendizaje automático

	Supervisado	No supervisado	Por refuerzo *no se ve en este curso*
Recibe	pares (x, y): la respuesta correcta	solo x, sin la etiqueta y	una recompensa por sus acciones
Aprende	predecir y para un x nuevo	grupos de observaciones	qué acción tomar en cada situación
Algoritmos	regresión lineal, logística, Ridge y Lasso	clustering: k-means, hierarchical clustering, DBSCAN	Q-learning, policy gradient

Jordan y Mitchell (2015). El supervisado es el más usado; el curso ve los dos primeros.

Aprendizaje supervisado y no supervisado

Supervisado	
Datos	pares (x, y): cada observación con su y
Objetivo	predecir y para un x nuevo
Algoritmos	regresión lineal (clase 04) logística (clase 06) Ridge y Lasso (clase 07)

No supervisado	
Datos	solo x, sin una respuesta y
Objetivo	encontrar grupos de observaciones parecidas
Algoritmos	clustering: k-means, hierarchical clustering y DBSCAN

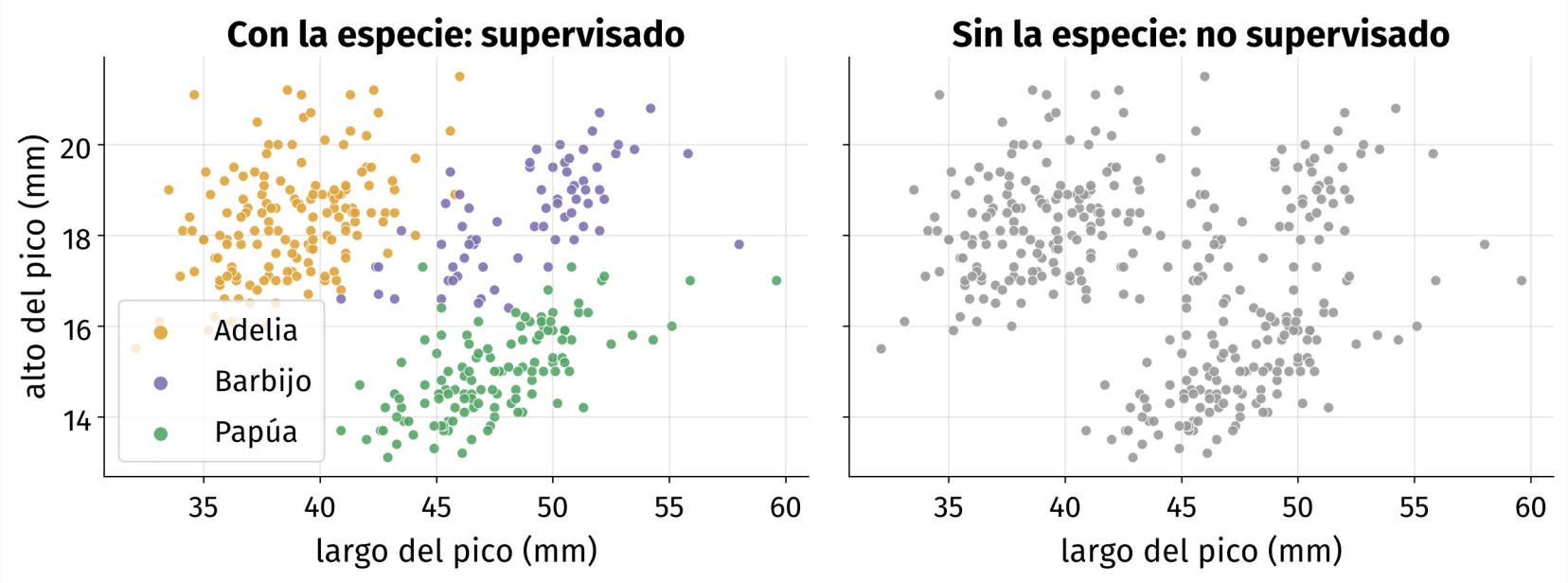
La distinción es de James et al. (2013). El tipo lo definen los datos y el objetivo, no el algoritmo: un mismo objetivo se puede lograr con distintos modelos. Todo lo visto hasta la clase 07 es supervisado.

Conjunto de datos: Palmer Penguins

especie	isla	largo_pico_mm	alto_pico_mm	largo_aleta_mm	masa_g
Adelia	Torgersen	39,1	18,7	181	3.750
Adelia	Torgersen	39,5	17,4	186	3.800
Barbijo	Dream	46,5	17,9	192	3.500
Barbijo	Dream	50,0	19,5	196	3.900
Papúa	Biscoe	46,1	13,2	211	4.500
Papúa	Biscoe	50,0	16,3	230	5.700

342 pingüinos adultos medidos entre 2007 y 2009 en el archipiélago Palmer, Antártida: 151 Adelia, 68 barbijo y 123 papúa; medidas en milímetros y gramos. Gorman et al. (2014) los midieron para estudiar las diferencias entre machos y hembras de cada especie. Hoy se usan las medidas y la especie.

Clasificación y clustering sobre los mismos datos

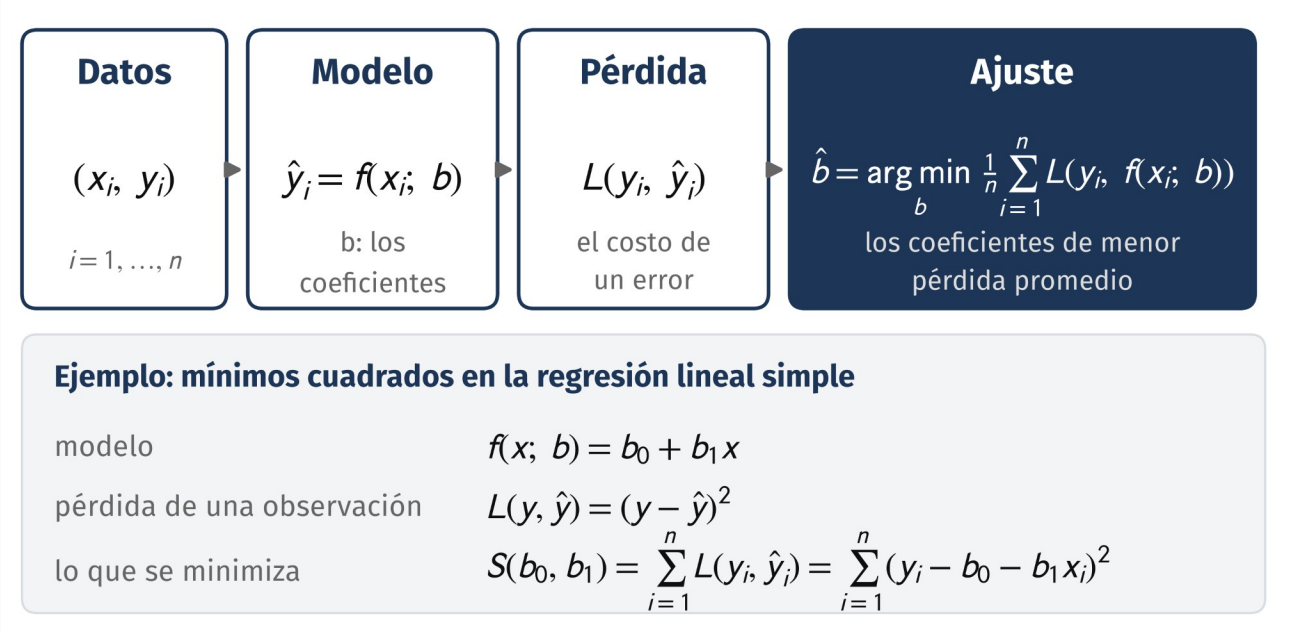


Con la especie como respuesta, el problema es supervisado: predecir la especie a partir de las medidas es un problema de clasificación. Sin la especie, el problema es no supervisado: se buscan grupos en las medidas y después se comparan con las especies.

2. La función de pérdida

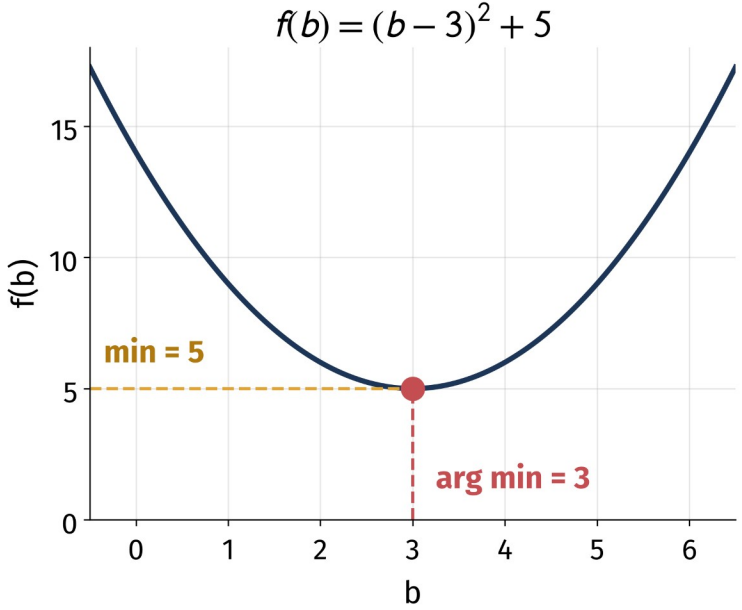
Definición, notación y ejemplos

Función de pérdida para el caso supervisado



Se define para el aprendizaje supervisado: cada observación tiene su respuesta y, y la pérdida compara la predicción ŷ con ella. L(y, ŷ) ≥ 0 es la pérdida de una observación; lo que se minimiza es su suma S en los datos de entrenamiento, o su promedio S/n, que da los mismos coeficientes. Cada modelo elige su L, y con ella qué errores importan (Hastie, Tibshirani y Friedman, 2009, sección 2.4).

Mínimo y argumento del mínimo (arg min)



$\min_b \; f(b) = 5$

el valor más bajo de la función

$\arg\min_b \; f(b) = 3$

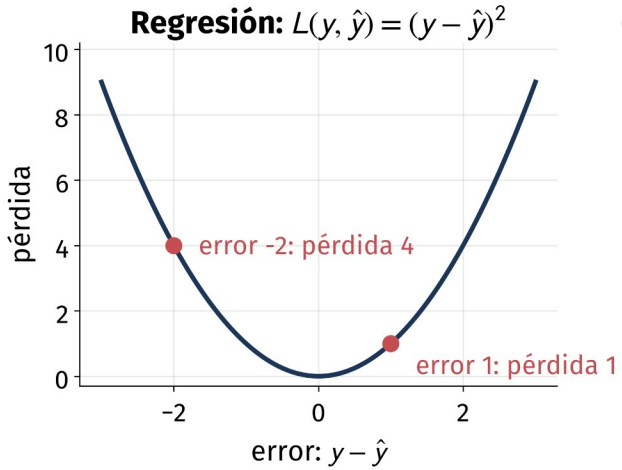
el b donde se alcanza

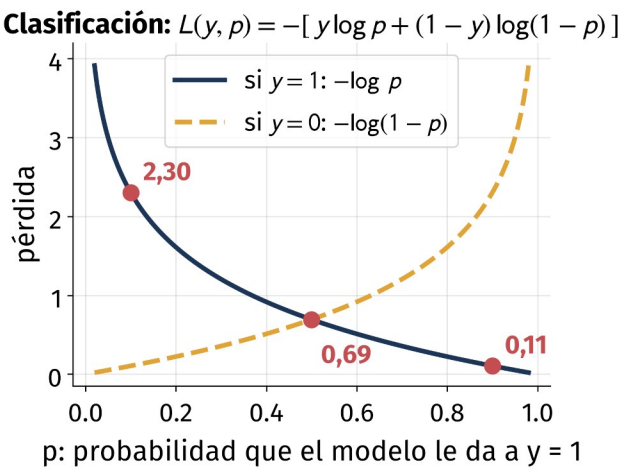
$f'(b) = 2(b-3) = 0 \;\Rightarrow\; b = 3$

derivar e igualar a cero

arg min se lee «el argumento que minimiza»: no es el valor más bajo de la función, sino el valor de la variable donde se alcanza. En mínimos cuadrados pasa lo mismo con la pérdida: el arg min son los coeficientes $\hat{b}$ y el mínimo es la pérdida que se logra con ellos.

Pérdida cuadrática y pérdida logística





Ajustar el modelo es elegir sus coeficientes para minimizar $\frac{1}{n} \sum_{i=1}^{n} L(y_i, \hat{y}_i)$ en los n datos de entrenamiento

Cuadrática: el error entra al cuadrado, así que un error del doble cuesta cuatro veces más. Logística: solo cuenta el término del valor real; con y = 1 la pérdida es −log p, y el modelo seguro y equivocado (p = 0,1) paga 2,30, más de tres veces lo que paga el que duda (p = 0,5).

Cálculo de la pérdida logística en un ejemplo

si fue un 1: $L = -\log p$ **(conviene una p alta)**

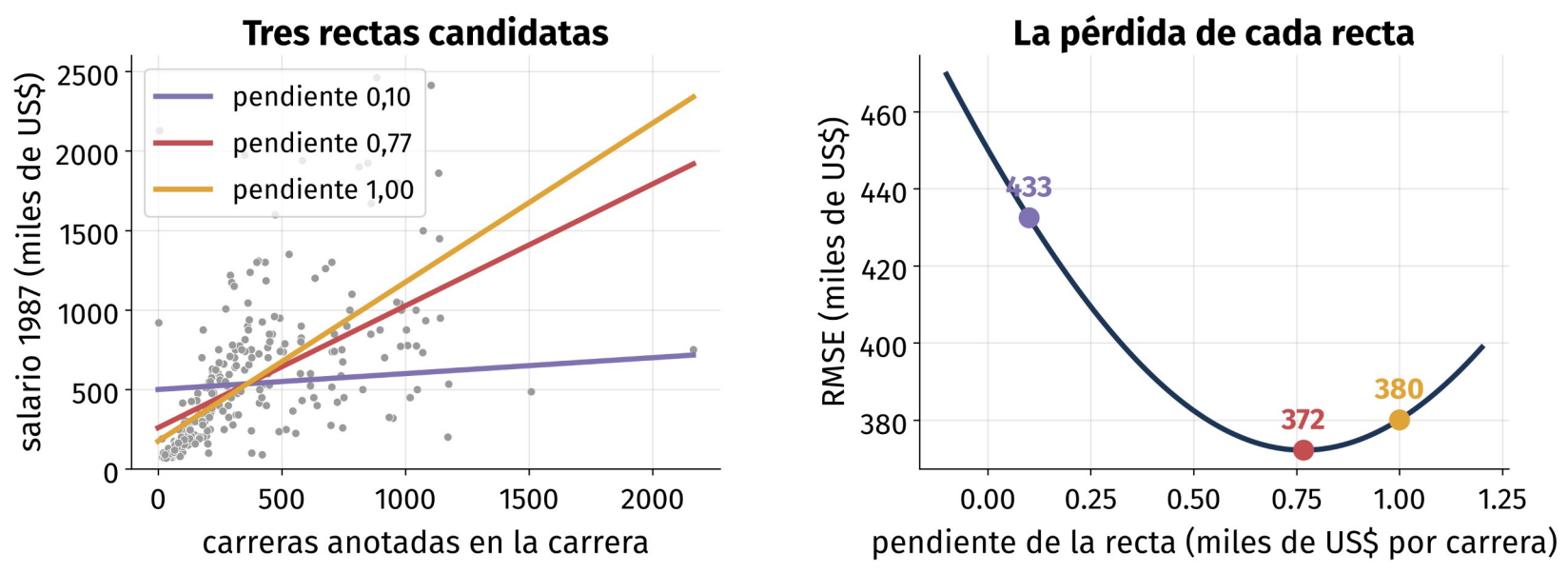
si fue un 0: $L = -\log(1 - p)$ **(conviene una p baja)**

cliente	p del modelo	qué pasó	cobro	pérdida
1	0,9	compró (1)	-log(0,9)	**0,11**
2	0,6	compró (1)	-log(0,6)	**0,51**
3	0,2	no compró (0)	-log(1 - 0,2)	**0,22**
4	0,7	no compró (0)	-log(1 - 0,7)	**1,20**

Total = 0,11 + 0,51 + 0,22 + 1,20 = 2,04 · promedio = 0,51

El modelo es uno solo, la curva en S, que le da una p a cada cliente. Las dos curvas de la slide anterior son estas dos reglas de cobro: los clientes 1 y 2 se cobran con la azul y los 3 y 4, con la amarilla. Ajustar la logística es buscar los coeficientes de la curva en S que hacen más chico el total.

Pérdida en función de la pendiente



Los 263 jugadores de la clase 07: salario contra carreras anotadas en la carrera. Cada pendiente da una recta y cada recta, un error. La de mínimos cuadrados (0,77) tiene el menor: RMSE 372, contra 433 y 380 de las otras dos.

Funciones de pérdida de uso frecuente

Método	Pérdida de una observación, L	Lo que se minimiza, S
Mínimos cuadrados	$(y - \hat{y})^2$	$\sum_i (y_i - \hat{y}_i)^2$
Desviación absoluta	$\lvert y - \hat{y} \rvert$	$\sum_i \lvert y_i - \hat{y}_i \rvert$
Regresión logística	$-[\, y \log p + (1 - y) \log(1 - p) \,]$	$\sum_i L(y_i, p_i)$
Ridge	$(y - \hat{y})^2$	$\sum_i (y_i - \hat{y}_i)^2 + \lambda \sum_j b_j^2$
Lasso	$(y - \hat{y})^2$	$\sum_i (y_i - \hat{y}_i)^2 + \lambda \sum_j \lvert b_j \rvert$

Ridge y Lasso usan la pérdida de mínimos cuadrados y le suman un castigo por los coeficientes grandes; λ fija cuánto pesa. Con la pérdida absoluta, la mejor predicción constante es la mediana y no el promedio: elegir L cambia lo que el modelo aprende (Hastie, Tibshirani y Friedman, 2009, sección 2.4).

Cómo se encuentran los coeficientes

Modelo	¿Tiene fórmula?	Cómo se encuentran los coeficientes
Mínimos cuadrados	Sí	$\hat{b} = (X^\top X)^{-1} X^\top y$
Ridge	Sí	$\hat{b} = (X^\top X + \lambda I)^{-1} X^\top y$
Regresión logística	No: las ecuaciones no son lineales en los coeficientes	Se itera: Newton-Raphson o variantes (`lbfgs` en scikit-learn)
Lasso	No: el valor absoluto no se puede derivar en cero	Se itera: *coordinate descent*, un coeficiente a la vez
k-means	No	Se itera: el algoritmo de Lloyd
Redes neuronales	No	Se itera: descenso de gradiente

Lo que decide si hay que iterar no es cuántos coeficientes hay, sino la forma de la pérdida. La cuadrática da ecuaciones lineales y se resuelve directo, con uno o con cien coeficientes. Cuando no hay fórmula, se parte de unos coeficientes y se mejoran paso a paso hasta que la pérdida ya no baja.

3. Reducción de dimensionalidad

Análisis de componentes principales (PCA)

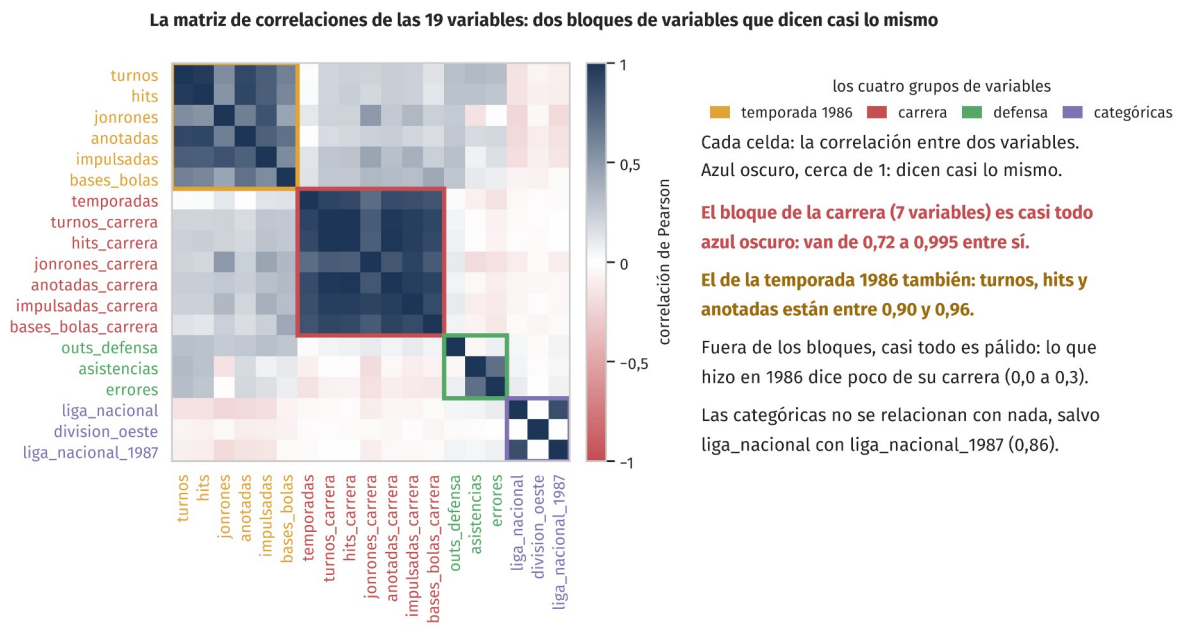
Conjunto de datos: jugadores de béisbol

jugador	salario	temporadas	turnos	hits	jonrones	turnos_ carrera	hits_ carrera
1	475,0	14	315	81	7	3.449	835
2	480,0	3	479	130	18	1.624	457
3	500,0	11	496	141	20	5.628	1.575
4	91,5	2	321	87	10	396	101
5	750,0	11	594	169	4	4.408	1.133

263 jugadores y 19 variables; la tabla muestra 7. El salario es lo que queremos predecir.

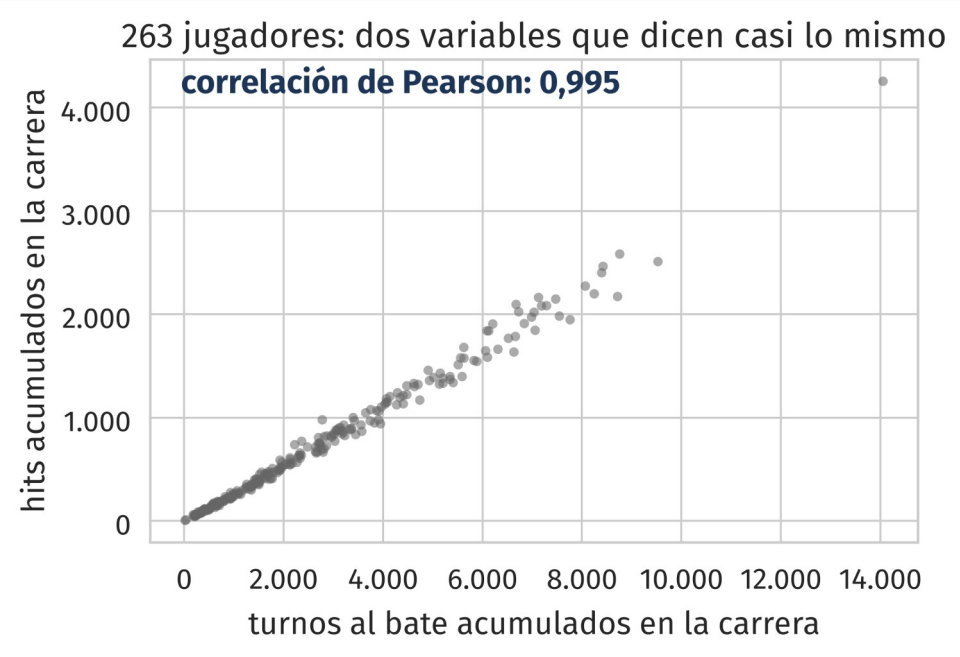
263 jugadores de las grandes ligas con su salario de 1987 y 19 variables de la temporada 1986 y de su carrera (James et al., 2013). PCA usa solo las 16 variables numéricas y no mira el salario; el salario vuelve al final de la sección.

Matriz de correlaciones



La matriz de la clase 07. Las 7 variables de la carrera se correlacionan entre 0,72 y 0,995; en el bloque de la temporada 1986, turnos, hits y anotadas están entre 0,90 y 0,96. Cada bloque dice casi lo mismo por dentro. La pregunta: ¿cuántas cosas distintas miden las 16 variables numéricas?

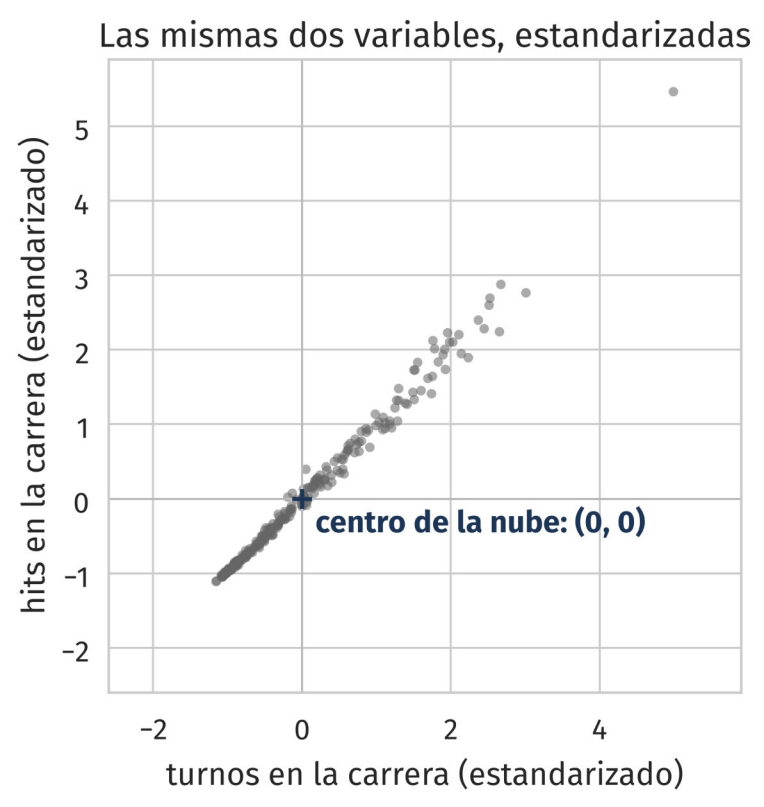
Dos variables altamente correlacionadas



¿Cuánto se pierde si las dos variables se reemplazan por una sola?

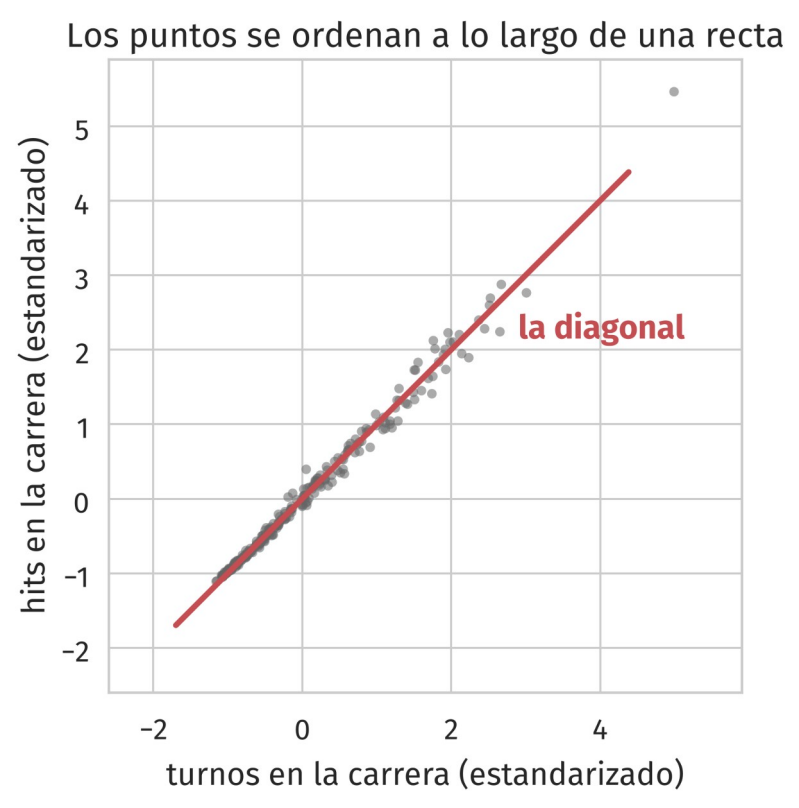
Cada punto es un jugador, con sus turnos al bate y sus hits acumulados en la carrera. Correlación 0,995: los puntos siguen una diagonal y saber una variable es saber casi la otra.

Estandarización de las variables



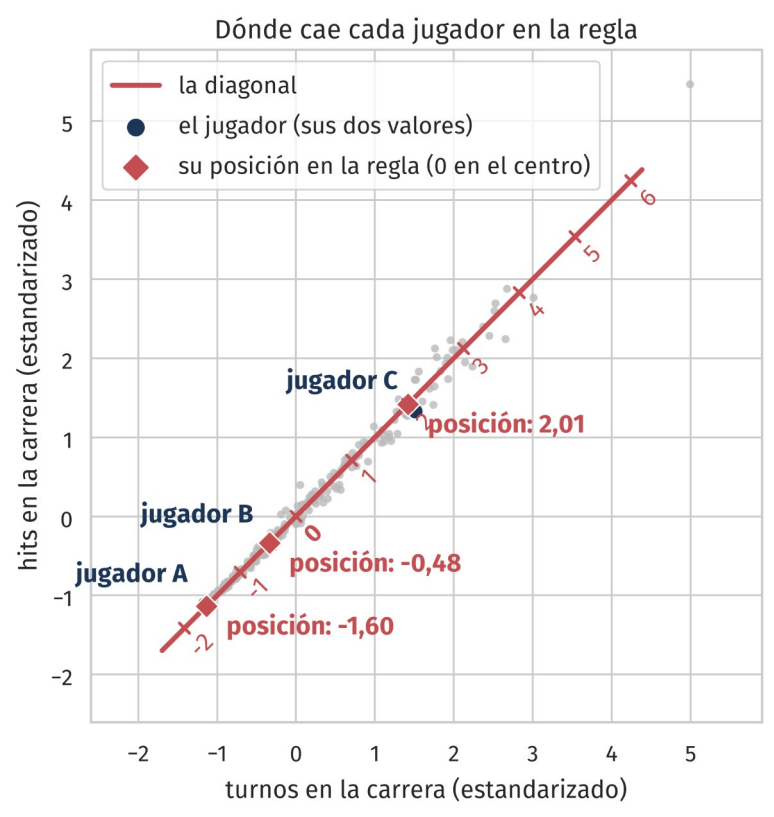
A cada variable se le resta su media y se divide por su desviación estándar (el puntaje z). Las dos quedan con media 0 y desviación 1, y el centro de la nube en el origen.

Dirección de máxima dispersión



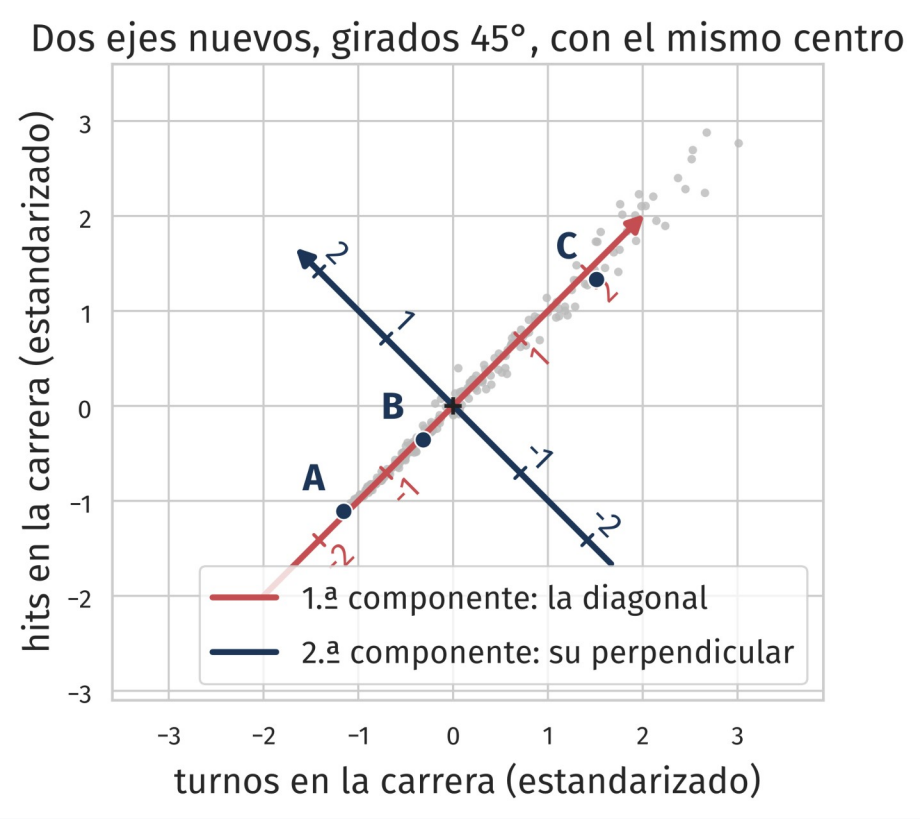
La recta pasa por el centro y sigue la nube de puntos. Es la dirección en que los jugadores se diferencian: hacia arriba y a la derecha, más carrera.

Proyección sobre la dirección principal



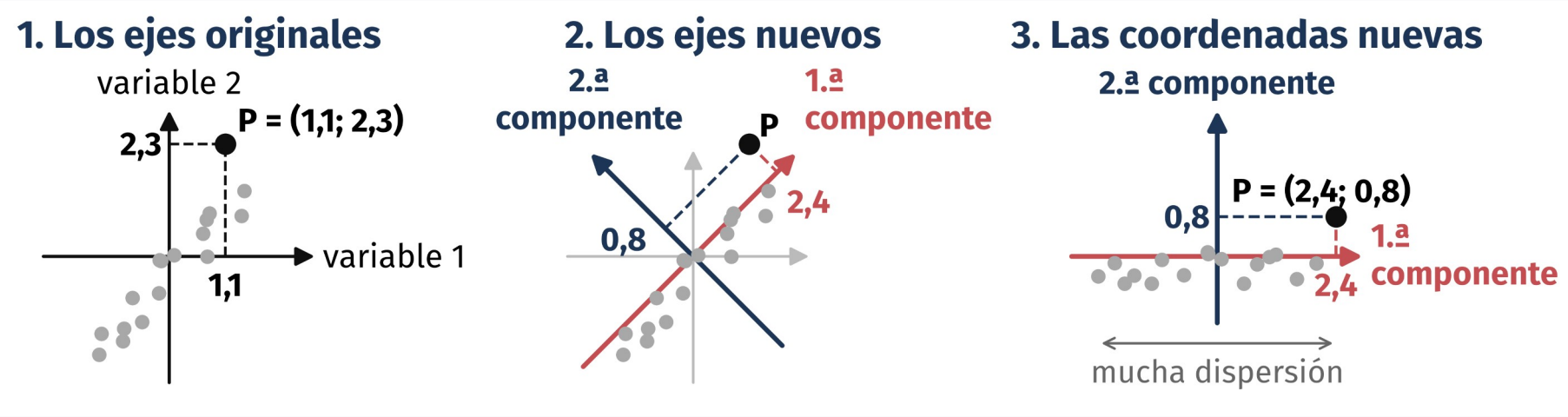
- **La diagonal es una regla: el 0 en el centro de la nube, en las unidades de los ejes, positivo hacia arriba y a la derecha.**
- A cada jugador se le marca el punto de la regla más cercano y se lee el número: A cae en −1,60, B en −0,48 y C en 2,01.
- Ese número es la variable nueva: reemplaza a los dos valores de cada jugador, sus turnos y sus hits.

Rotación de ejes: las componentes principales



- **Un cambio de sistema de referencia: en vez de turnos y hits, dos ejes nuevos girados 45°, la diagonal y su perpendicular, con el mismo centro.**
- Cada jugador tiene dos coordenadas nuevas. A pasa de (−1,16; −1,11) a (−1,60; 0,03); C, de (1,51; 1,33) a (2,01; −0,13).
- La segunda coordenada es casi 0 para todos: es lo que se descarta al quedarse con un solo número.
- El primer eje se llama primera componente principal; el segundo, segunda.

Cambio de coordenadas



Los mismos puntos, medidos con otros ejes: P pasa de (1,1; 2,3) a (2,4; 0,8). Con la figura girada se ve que casi toda la dispersión queda en la 1.ª componente.

La componente como combinación lineal

primera componente = 0,707 × z(turnos_carrera) + 0,707 × z(hits_carrera)

z = valor estandarizado: (valor − media) / desviación estándar

jugador	turnos carrera	hits carrera	z turnos	z hits	0,707 × z turnos + 0,707 × z hits	1.ª componente
A	19	4	-1,16	-1,11	0,707 × (-1,16) + 0,707 × (-1,11)	**-1,60**
B	1.931	491	-0,32	-0,36	0,707 × (-0,32) + 0,707 × (-0,36)	**-0,48**
C	6.100	1.583	1,51	1,33	0,707 × (1,51) + 0,707 × (1,33)	**2,01**

Con 16 variables es la misma cuenta, con 16 términos:

0,33 × z(turnos_carrera) + 0,33 × z(hits_carrera) + … + 0,20 × z(turnos) + … − 0,01 × z(errores)

Con dos variables, los pesos son 0,707 y 0,707: apuntan en la diagonal, la dirección en que los puntos más se dispersan. Con 16 variables los pesos cambian, pero siempre los fijan los datos.

Proporción de varianza explicada

$$\mathrm{Var}(z) = \frac{1}{n} \sum_{i=1}^{n} (z_i - \bar{z})^2$$

La variación total es la suma de las varianzas de las coordenadas, y no cambia al girar los ejes

varianza = dispersión al cuadrado

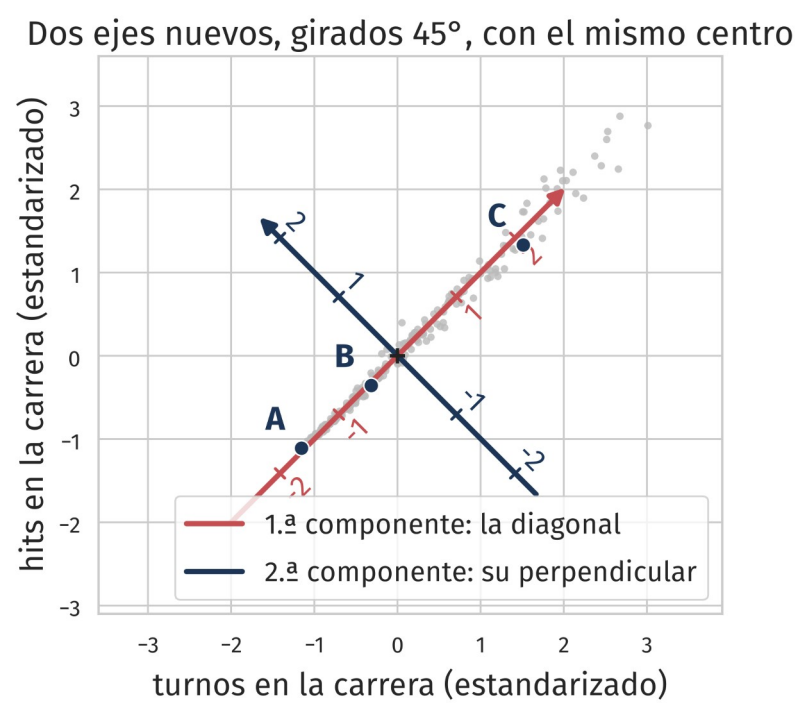
ejes originales	turnos (z): 1,000	+	hits (z): 1,000	=	total: 2,000
ejes nuevos	1.ª coordenada: 1,995	+	2.ª coordenada: 0,005	=	total: 2,000
(dispersiones	$1{,}412^2 = 1{,}995$		$0{,}070^2 = 0{,}005$)		

parte de la 1.ª: 1,995 / 2,000 = 99,8% **parte de la 2.ª: 0,005 / 2,000 = 0,2%**

Quedarse con la posición sobre la diagonal conserva el 99,8% de la variación de los datos y pierde el 0,2%. Sale de las varianzas: las dos variables estandarizadas tienen 1 y 1 (total 2); las dos coordenadas nuevas reparten esa misma variación, 1,995 y 0,005; 1,995 / 2 = 99,8% y 0,005 / 2 = 0,2%. Con 16 variables, la variación total es 16 y cada componente se lleva una parte.

El análisis de componentes principales

- **Busca la dirección en que los datos más se dispersan (la primera componente), luego la que más se dispersa entre las perpendiculares (la segunda), y así hasta tener tantas como variables.**
- Cada componente es una variable nueva: un puntaje ponderado de las originales, con una parte de la variación total.



Usos del análisis de componentes principales

- **Para mirar los datos: con 16 variables hay 120 gráficos de dispersión posibles; con dos componentes, uno solo.**
- Para entregarle a un modelo menos entradas: las componentes reemplazan a un grupo de variables que se repiten.
- Para entender de qué está hecha la variación: qué variables se mueven juntas y cuánto pesa cada grupo.
- Lo que no hace: no elige variables ni las descarta, y no sabe qué se quiere predecir.

Los pesos de las tres componentes

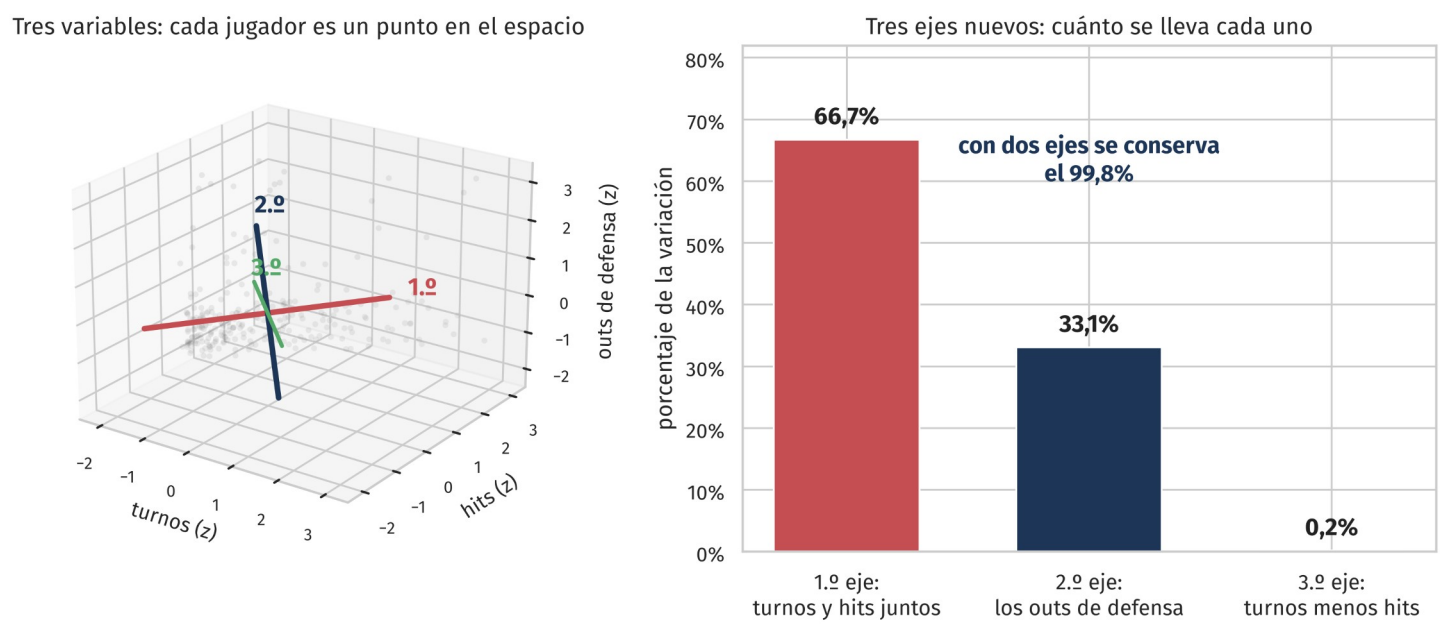
1.ª componente = 0,704 × z(turnos) + 0,705 × z(hits) + 0,085 × z(outs)
2.ª componente = −0,067 × z(turnos) − 0,053 × z(hits) + 0,996 × z(outs)
3.ª componente = 0,707 × z(turnos) − 0,707 × z(hits) + 0,010 × z(outs)

jugador	turnos carrera	hits carrera	outs defensa	z turnos	z hits	z outs	componentes 1.ª	componentes 2.ª	componentes 3.ª
A	19	4	692	−1,16	−1,11	1,44	**−1,47**	**1,57**	**−0,02**
B	1.931	491	222	−0,32	−0,36	−0,25	**−0,50**	**−0,20**	**0,03**
C	6.100	1.583	316	1,51	1,33	0,09	**2,01**	**−0,08**	**0,13**

A en la 2.ª componente: −0,067 × (−1,16) − 0,053 × (−1,11) + 0,996 × 1,44 = 1,57

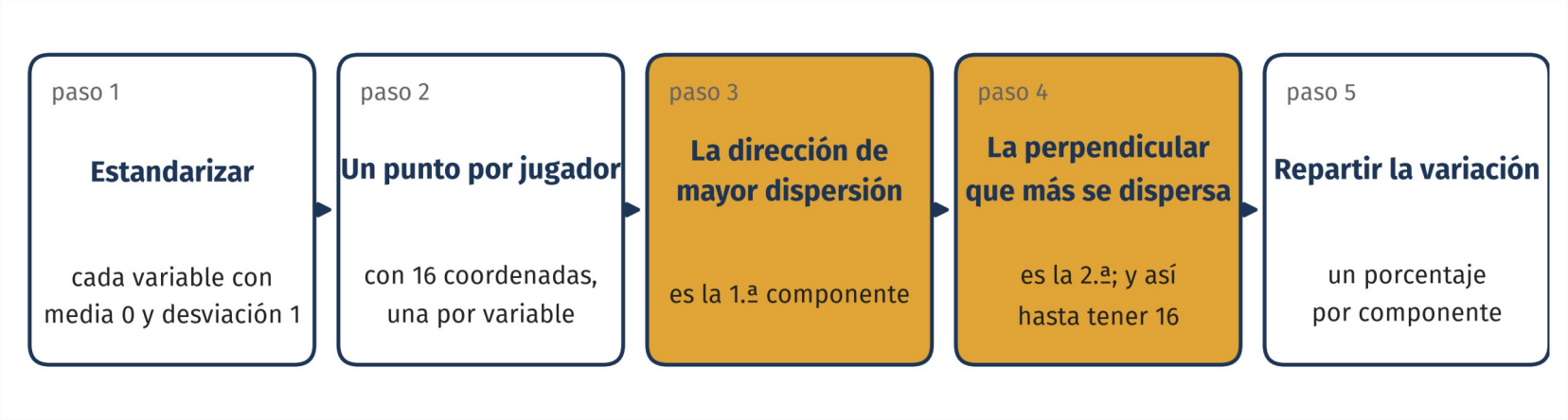
La 1.ª componente pesa turnos y hits casi por igual, la 2.ª es casi solo los outs de defensa y la 3.ª, turnos menos hits. A tiene turnos y hits bajo el promedio y muchos outs: por eso su 1.ª componente es negativa (−1,47) y su 2.ª, alta (1,57).

Componentes principales con tres variables



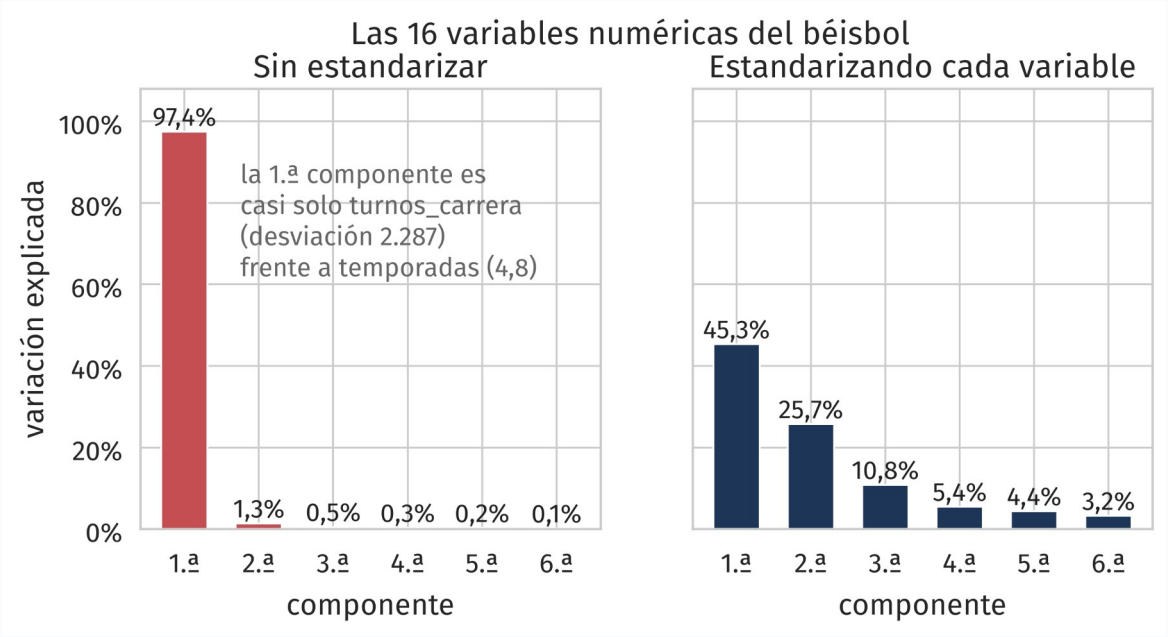
Siempre salen tantos ejes como variables. Aquí se agrega una tercera que no tiene que ver con las otras dos, los outs de defensa: el primer eje junta turnos y hits (66,7%), el segundo es la defensa (33,1%) y el tercero, la diferencia entre turnos y hits, casi nada (0,2%). Con dos ejes se conserva el 99,8% y el tercero se descarta. Si las tres variables dijeran cosas distintas, harían falta los tres.

Componentes principales con 16 variables



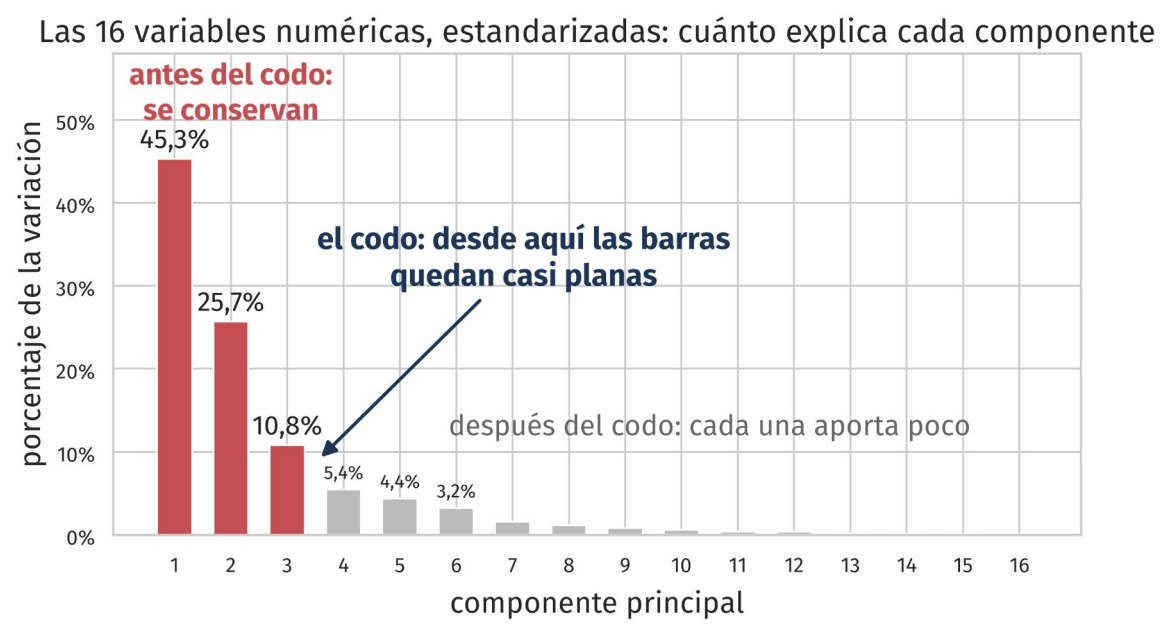
Con 16 variables la nube no se puede dibujar, pero el cálculo es el mismo: cada jugador es un punto con 16 coordenadas, se busca la dirección de mayor dispersión, luego la perpendicular que más se dispersa, y así hasta tener 16 componentes, ordenadas de la que más variación conserva a la que menos.

Efecto de la estandarización en PCA



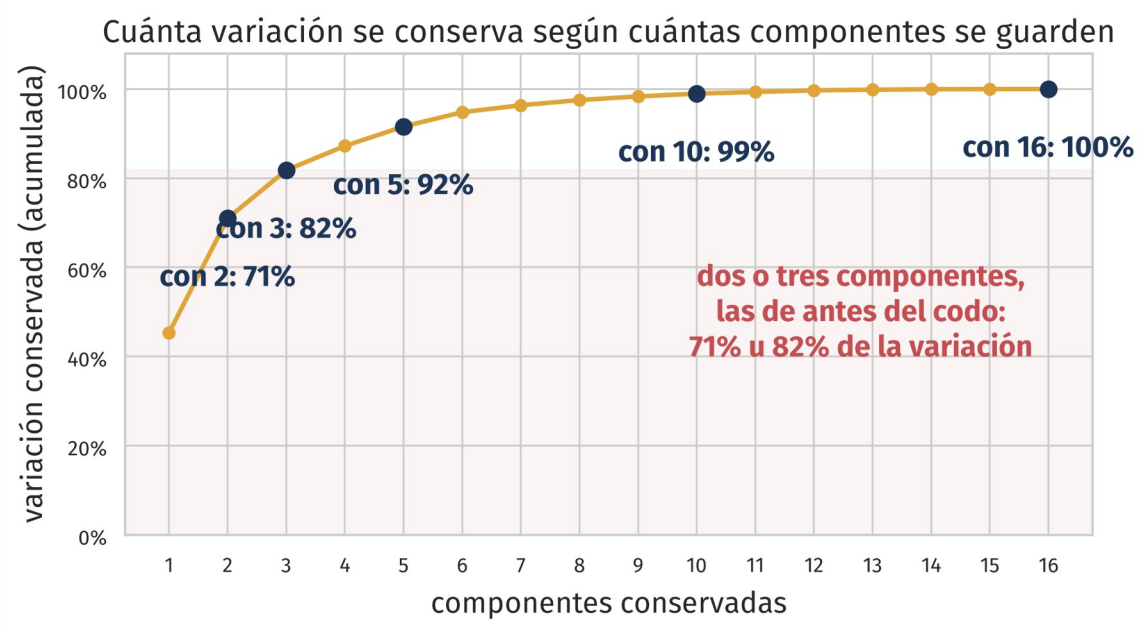
Con las variables en sus unidades, la primera componente es casi solo turnos_carrera, porque su desviación estándar (2.287) es 476 veces la de temporadas (4,8): PCA sigue a la variable que más varía en sus unidades. Con cada variable estandarizada (media 0 y desviación 1, como en la clase 07) todas parten iguales. Se estandariza salvo que las variables compartan unidad (James et al., 2013).

Varianza explicada por componente



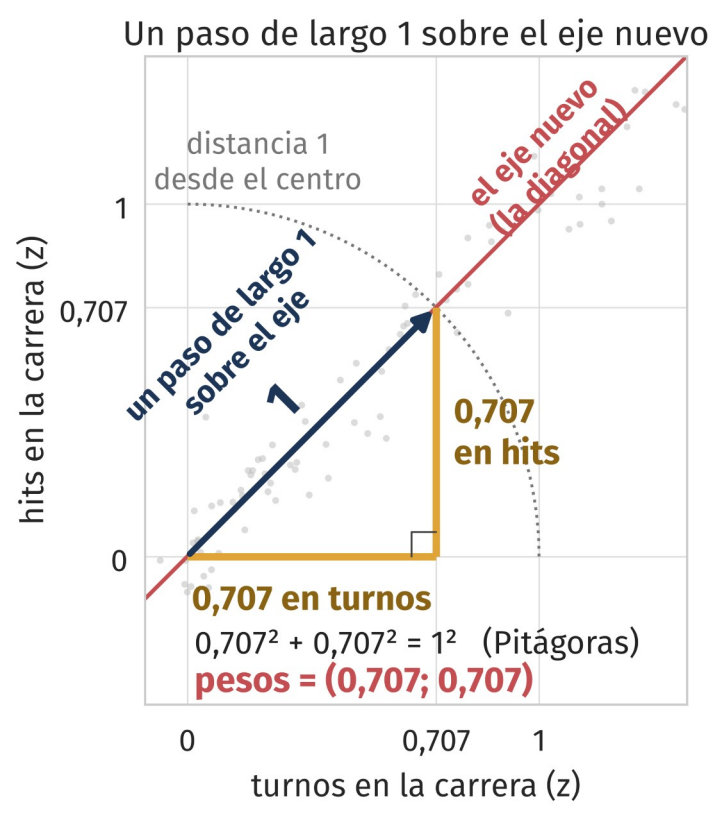
Las barras caen fuerte al principio (45,3%, 25,7%, 10,8%) y desde la cuarta quedan casi planas (5,4%, 4,4%, 3,2%...). Ese quiebre es el codo. Se conservan las componentes de antes del codo, que son las que explican mucho cada una: aquí, dos o tres.

Elección del número de componentes



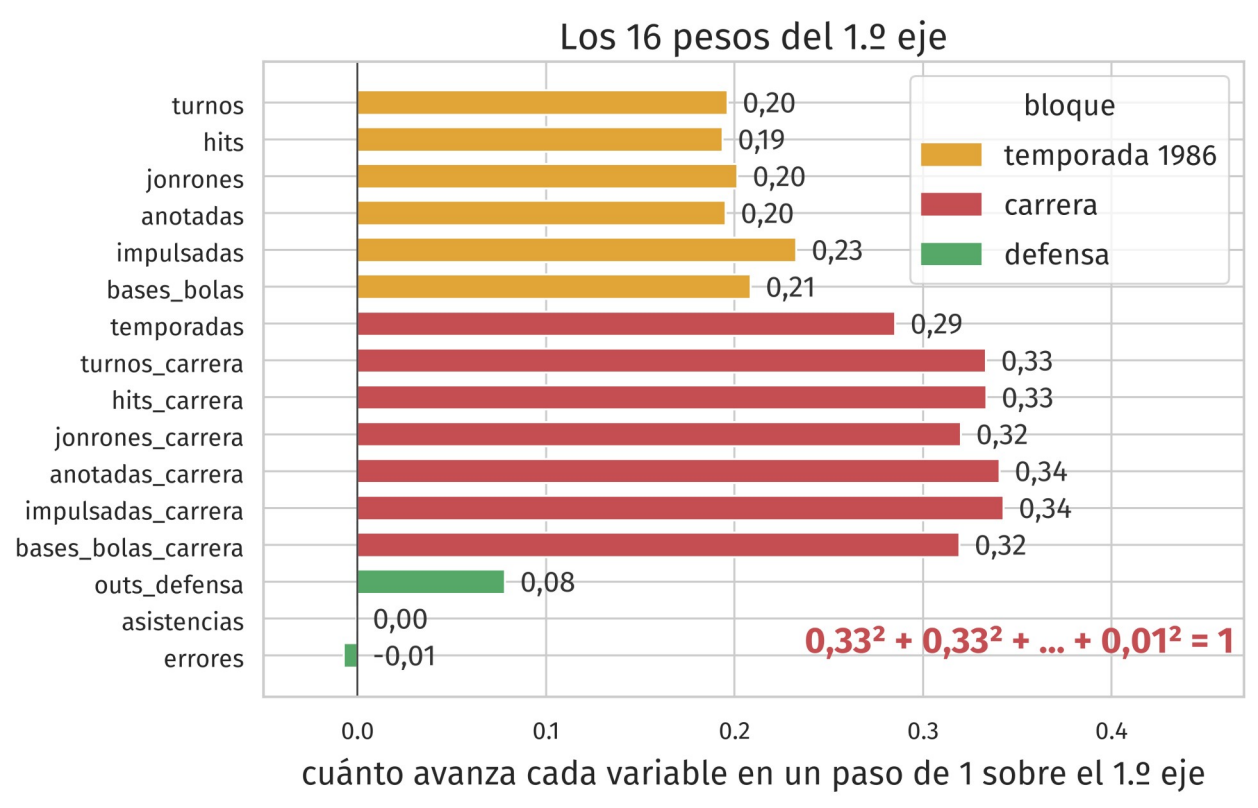
El acumulado dice cuánto se conserva con cada cantidad: con dos componentes, el 71%; con tres, el 82%; con cinco, el 92%; con diez, el 99%. Conservar diez de dieciséis casi no reduce nada: el codo sugiere dos o tres. No hay una regla objetiva: depende de cuánta variación se acepte perder y para qué se usen las componentes (James et al., 2013).

Los pesos (loadings) con dos variables



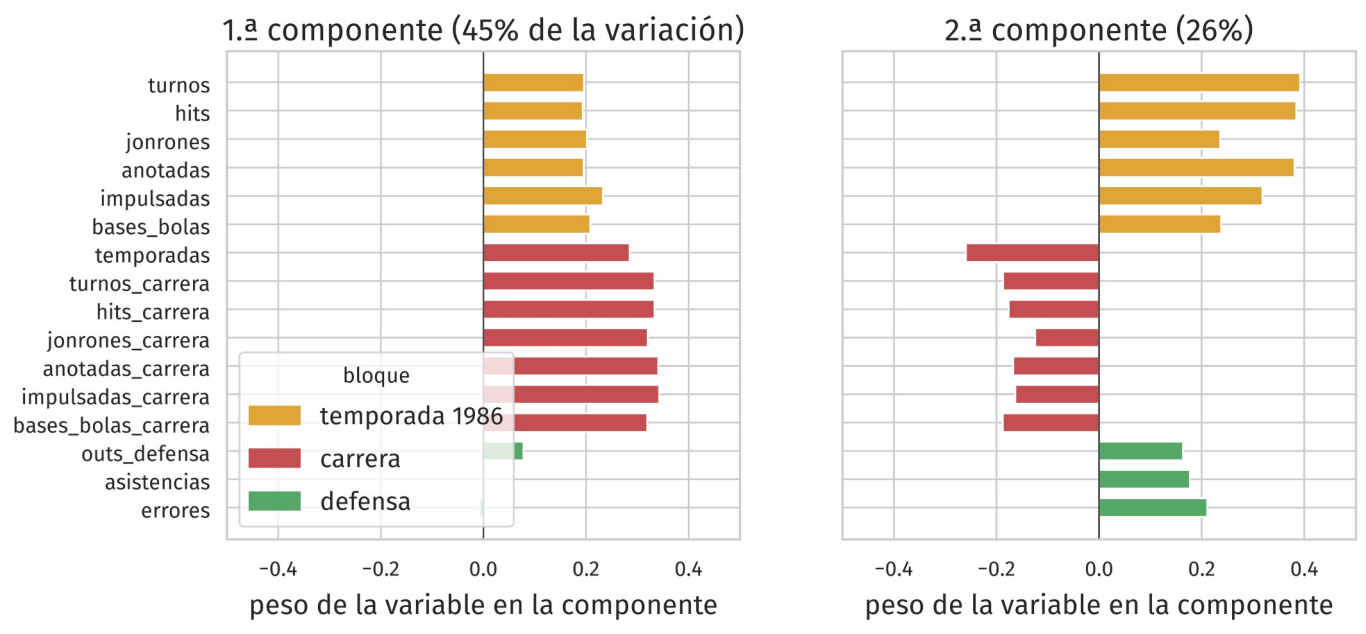
- **La flecha azul es un paso de largo 1 sobre el eje nuevo: su punta queda a distancia 1 del centro.**
- En ese paso, turnos avanza 0,707 y hits avanza 0,707: los dos catetos del triángulo ($0{,}707^2 + 0{,}707^2 = 1^2$).
- Esos dos números son los pesos de la primera componente: cuánto avanza cada variable en un paso de 1 sobre el eje.

Pesos de la primera componente con 16 variables



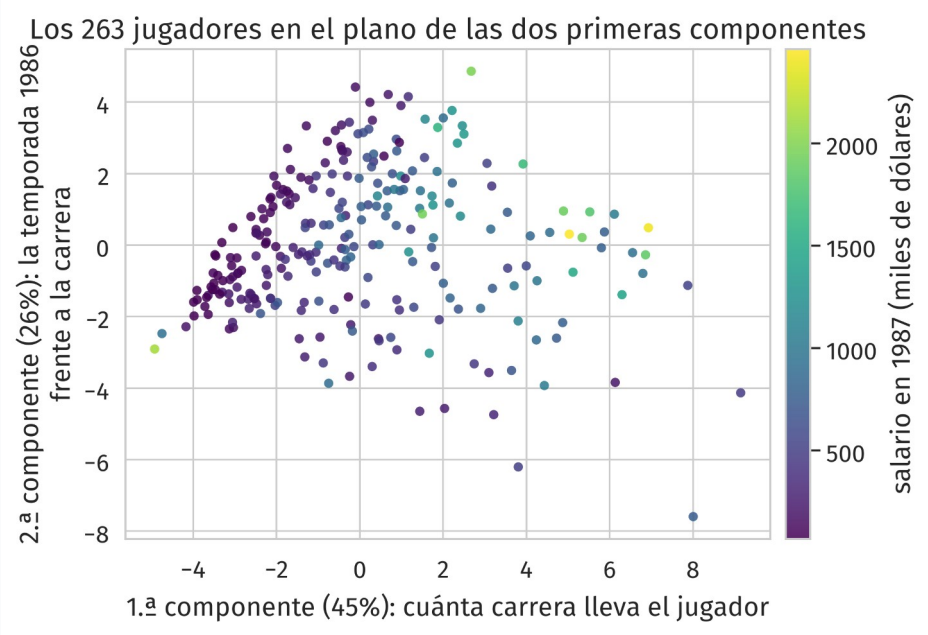
Lo mismo en 16 dimensiones, donde ya no se puede dibujar: un paso de 1 sobre el primer eje avanza 0,33 en turnos_carrera, 0,33 en hits_carrera, 0,20 en turnos de 1986, 0,08 en outs de defensa y casi nada en errores. Son dieciséis catetos cuyos cuadrados suman 1.

Interpretación de las componentes



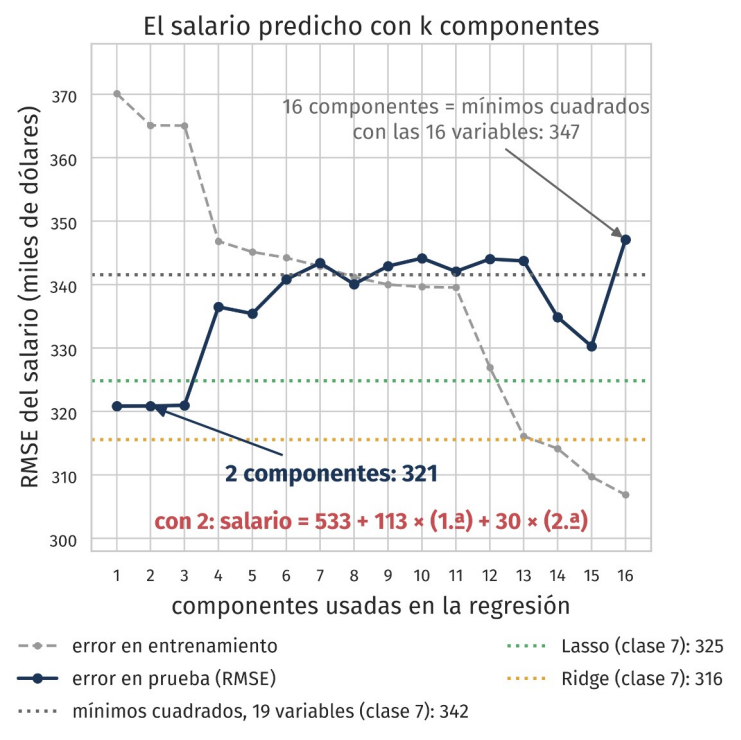
1.ª componente: la carrera pesa 0,29 a 0,34, la temporada 1986 la mitad (0,19 a 0,23) y la defensa nada: cuánta carrera lleva el jugador. 2.ª: la temporada positiva (0,24 a 0,39) y la carrera negativa (−0,12 a −0,26): quien rindió este año más de lo que dice su carrera.

Proyección en las dos primeras componentes



Cada punto es un jugador y el color, su salario de 1987, que PCA no usó. El salario crece hacia la derecha (correlación 0,64 con la primera componente) y casi nada hacia arriba (0,11 con la segunda): la carrera pesa en el salario; la última temporada, poco. Dos ejes muestran lo que en 16 variables había que leer coeficiente por coeficiente.

Regresión sobre componentes principales



- **Con el reparto de la clase 07 (131 y 132 jugadores), el salario se predice con las primeras k componentes. Con 1, 2 o 3, el error de prueba es 321.**
- Mínimos cuadrados con las mismas 16 variables da 347 y con las 19, 342; Lasso 325 y Ridge 316.
- Dos números por jugador predicen casi tan bien como el mejor modelo de la clase 07 (en 20 repartos le gana a mínimos cuadrados en 17): es la regresión sobre componentes principales (James et al., 2013, sección 6.3.1).

Ojo: aquí k se eligió mirando el error de prueba, y eso es usar la prueba para entrenar. Lo correcto es elegir k con validación cruzada en el entrenamiento: con la regla de un error estándar (James et al., 2013, sección 6.1.3) sale k = 1, con el mismo 321.

Cuándo aplicar PCA

- **Con muchas variables numéricas que se correlacionan en bloques, como en la matriz de la clase 07.**
- Para mirar los datos en un plano, o para entregarle a un modelo menos entradas.
- No aporta si las variables son pocas y distintas entre sí: no hay nada que resumir.

Limitaciones de PCA

- **Las variables categóricas (las dummies de la clase 06) no entran bien en el cálculo: se dejan fuera o se tratan aparte.**
- Las componentes no siempre tienen una lectura clara. Aquí sí (la carrera; la temporada frente a la carrera), pero no está garantizado.

Implementación de PCA en scikit-learn

```
Z = StandardScaler().fit_transform(X)  # 1. estandarizar
pca = PCA().fit(Z)                     # 2. las componentes
pca.explained_variance_ratio_          # 3. cuánto explica
pca.components_                        # 4. los pesos
puntajes = pca.transform(Z)            # 5. los puntajes
```

- El ejemplo completo, con las tablas de pesos y los gráficos de la clase, está en el notebook 08_aprendizaje_no_supervisado.ipynb.

Recreo

15 minutos · al volver: clustering con k-means

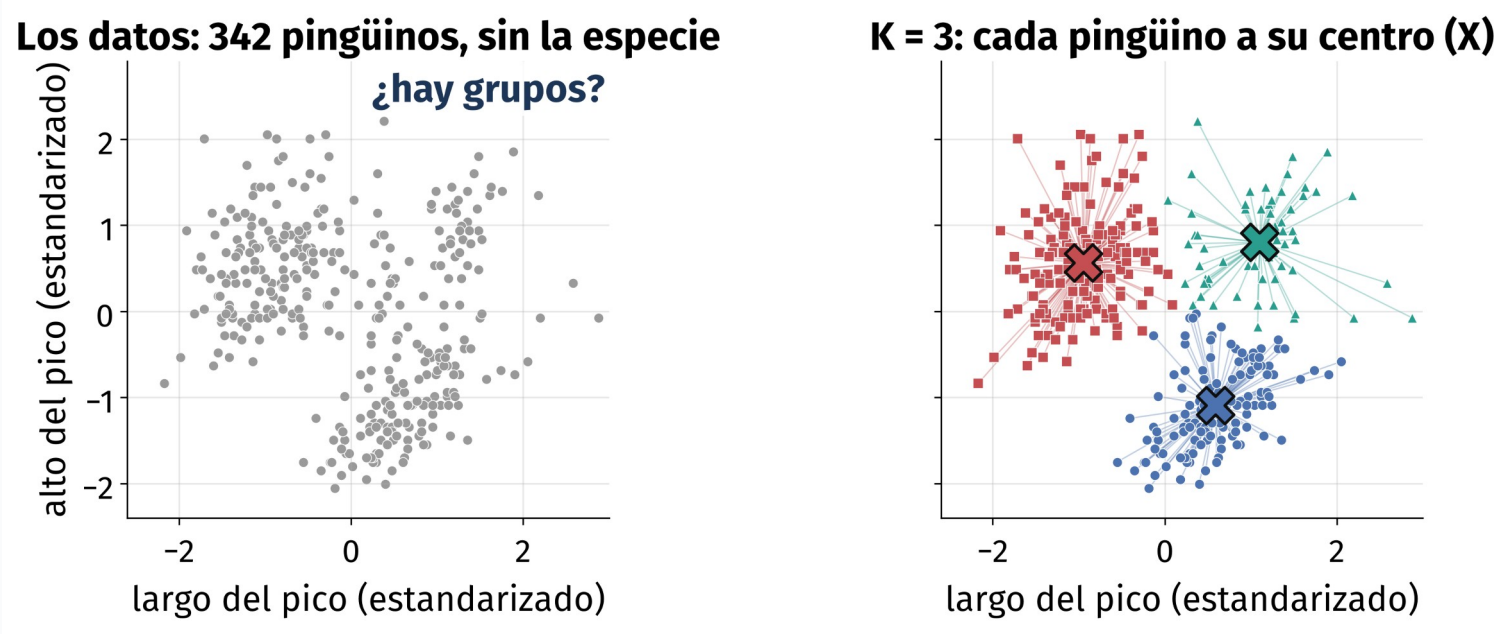
Bloque 2 · Clustering con k-means

Grupos de pingüinos con k-means

4. Clustering con k-means

Clustering de los pingüinos sin usar la especie

El problema del clustering



Dos medidas del pico, estandarizadas, sin la especie. k-means separa los puntos en K grupos, cada uno con su centro (la X), de modo que la suma de las distancias al cuadrado de cada pingüino a su centro sea la menor posible.

Función objetivo de k-means

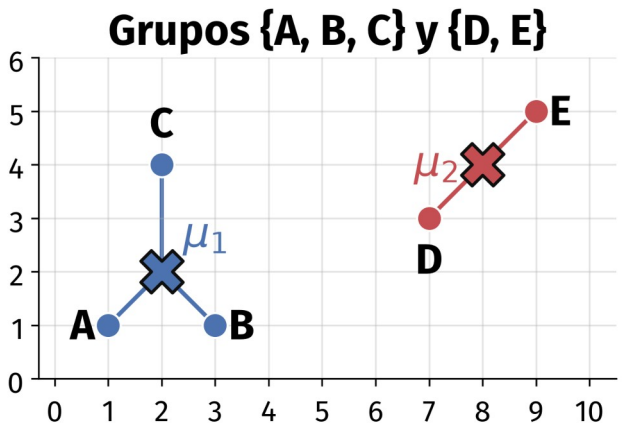
$$W = \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

K	el número de grupos, que se fija antes
C_k	las observaciones del grupo k; cada una está en un solo grupo
x_i	la observación i, con sus p variables estandarizadas
μ_k	el centro del grupo k: el promedio de sus observaciones
$\|x_i - \mu_k\|^2$	la distancia al cuadrado: $\sum_{j=1}^{p}(x_{ij} - \mu_{kj})^2$

W es la variación dentro de los grupos: la suma, en todos los grupos, de las distancias al cuadrado de cada observación al centro de su grupo. k-means busca los grupos con el menor W (James et al., 2013). El nombre es de MacQueen (1967); el algoritmo, de Lloyd (1982).

Nota: W es el equivalente a la función de pérdida en k-means, y algunos libros la llaman así (Hastie et al., 2009, sección 14.3).

Cálculo de W en un ejemplo pequeño



1. Los centros: el promedio de cada variable en el grupo

μ_1 = ((1 + 3 + 2) / 3, (1 + 1 + 4) / 3) = (2, 2)

μ_2 = ((7 + 9) / 2, (3 + 5) / 2) = (8, 4)

2. La distancia² de cada punto a su centro: $\|x_i - \mu_k\|^2$

punto	centro	variable 1 + variable 2	distancia²
A (1, 1)	μ_1	$(1-2)^2 + (1-2)^2$	**2**
B (3, 1)	μ_1	$(3-2)^2 + (1-2)^2$	**2**
C (2, 4)	μ_1	$(2-2)^2 + (4-2)^2$	**4**
D (7, 3)	μ_2	$(7-8)^2 + (3-4)^2$	**2**
E (9, 5)	μ_2	$(9-8)^2 + (5-4)^2$	**2**

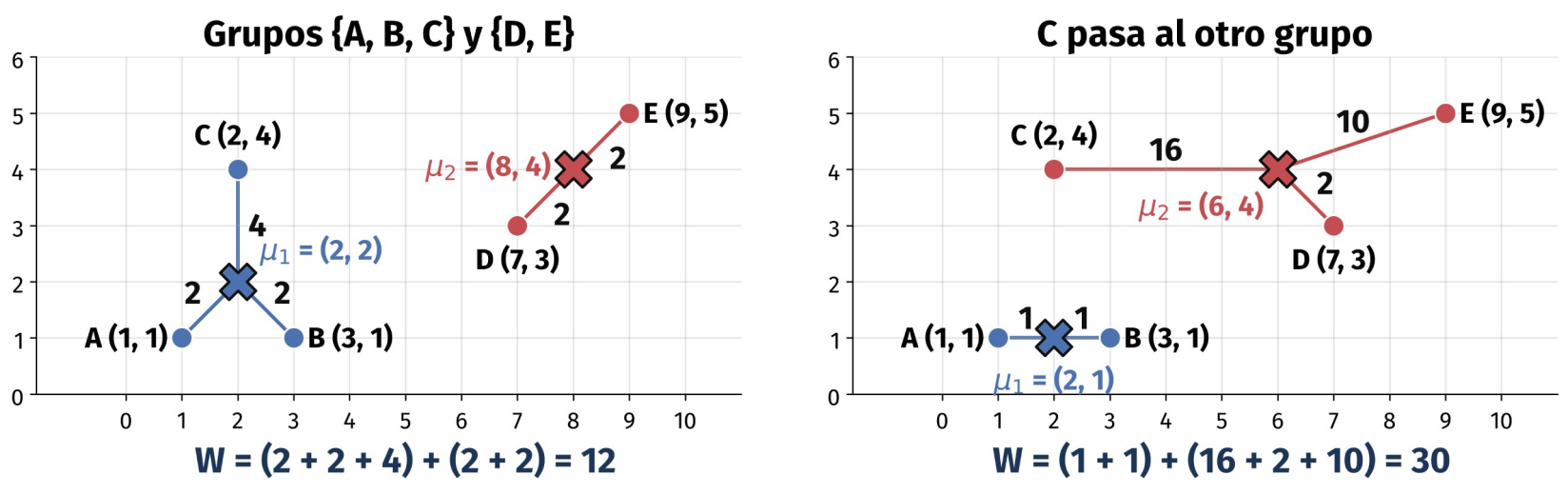
3. Se suman todas, grupo por grupo:

$$W = \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

k recorre los grupos;
i, los puntos de cada grupo

W = (2 + 2 + 4) + (2 + 2) = 8 + 4 = 12

Comparación de dos formas de agrupar



Los números sobre las líneas son las distancias² de cada punto a su centro. Si C pasa al otro grupo, los centros se mueven a (2, 1) y (6, 4) y W sube de 12 a 30: C queda lejos de su nuevo centro y lo aleja de D y E. k-means busca la forma de agrupar con el menor W.

El algoritmo de Lloyd

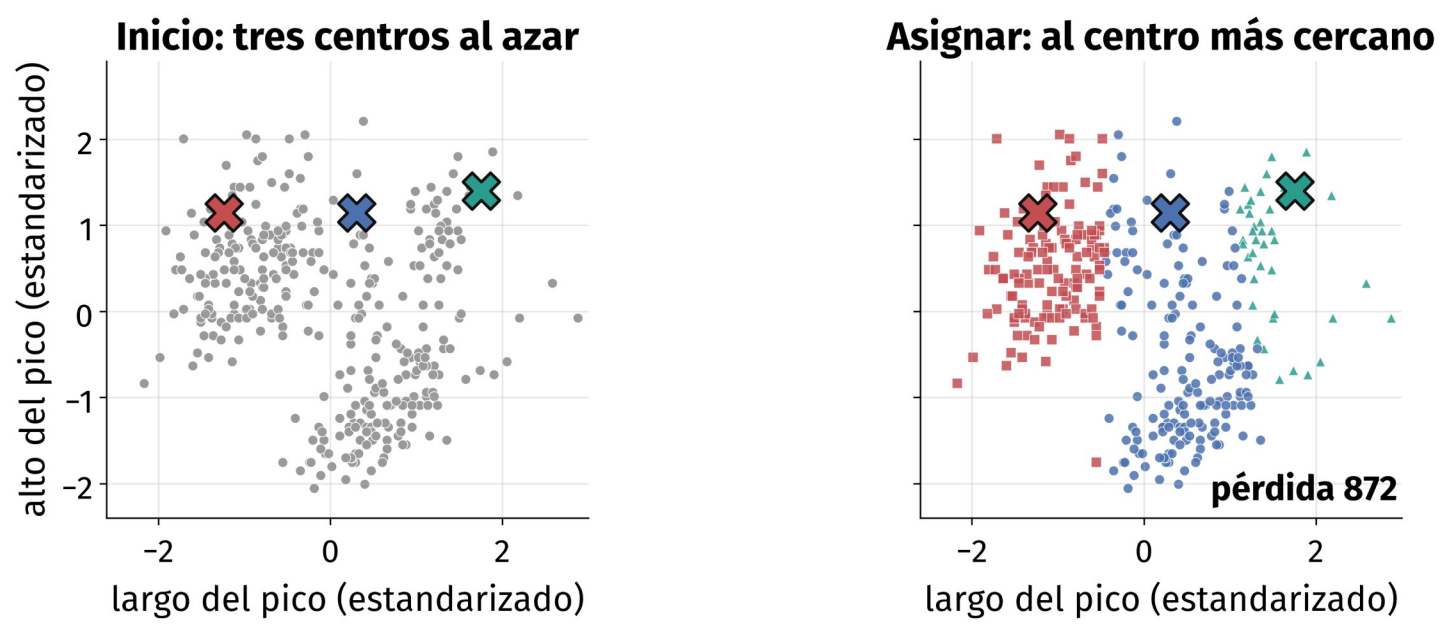
Algoritmo 1 k-means (algoritmo de Lloyd)

Entrada: datos estandarizados $x_1, \ldots, x_n$ y el número de grupos K
Salida: grupos $C_1, \ldots, C_K$ y sus centros $\mu_1, \ldots, \mu_K$

1: Elegir K observaciones al azar como centros $\mu_1, \ldots, \mu_K$
2: **repetir**
3: **para** $i = 1, \ldots, n$ **hacer** ▷ asignar
4: $c_i \leftarrow \arg\min_k \|x_i - \mu_k\|^2$
5: **fin para**
6: **para** $k = 1, \ldots, K$ **hacer** ▷ mover
7: $C_k \leftarrow \{\, i : c_i = k \,\}$
8: $\mu_k \leftarrow \dfrac{1}{|C_k|} \sum_{i \in C_k} x_i$
9: **fin para**
10: **hasta que** las asignaciones $c_1, \ldots, c_n$ no cambian
11: **devolver** los grupos, los centros y la pérdida $\displaystyle\sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \mu_k\|^2$

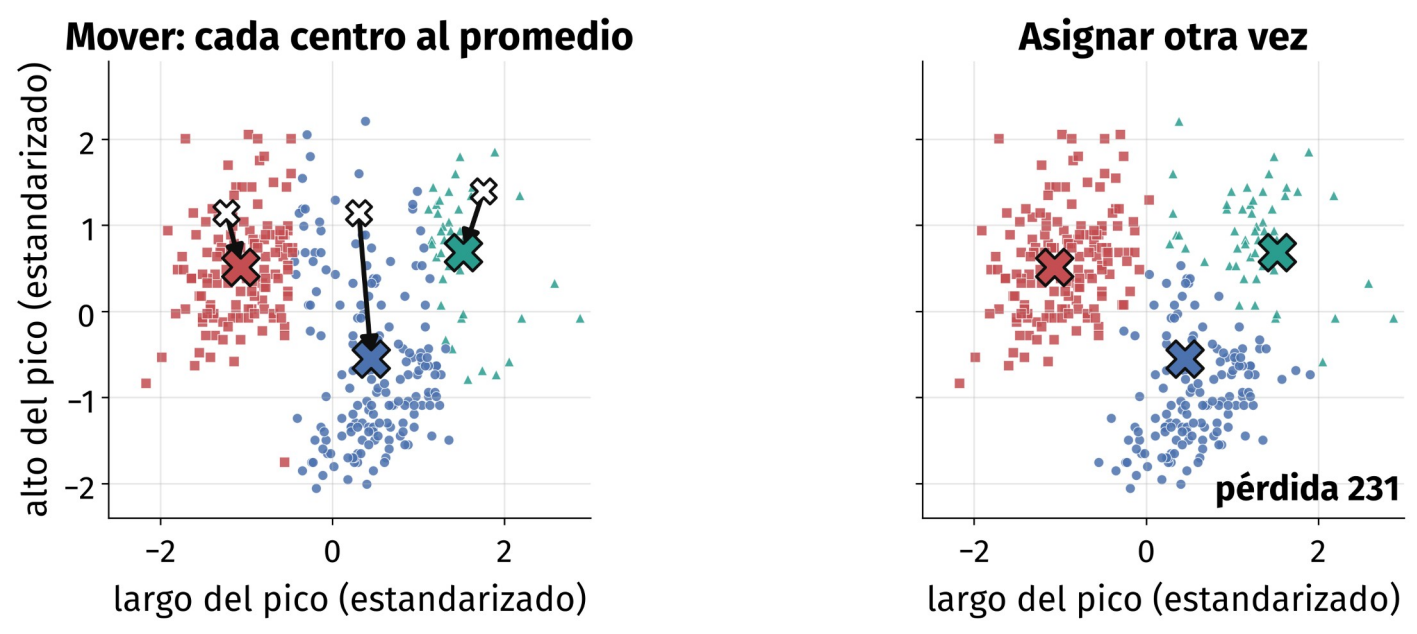
Se repiten dos pasos: asignar cada observación al centro más cercano y mover cada centro al promedio de su grupo. Cada ronda baja la pérdida o la deja igual (James et al., 2013, Algoritmo 10.1). El libro parte asignando los grupos al azar; desde el paso de mover, es lo mismo.

Ejemplo: inicialización de los centros



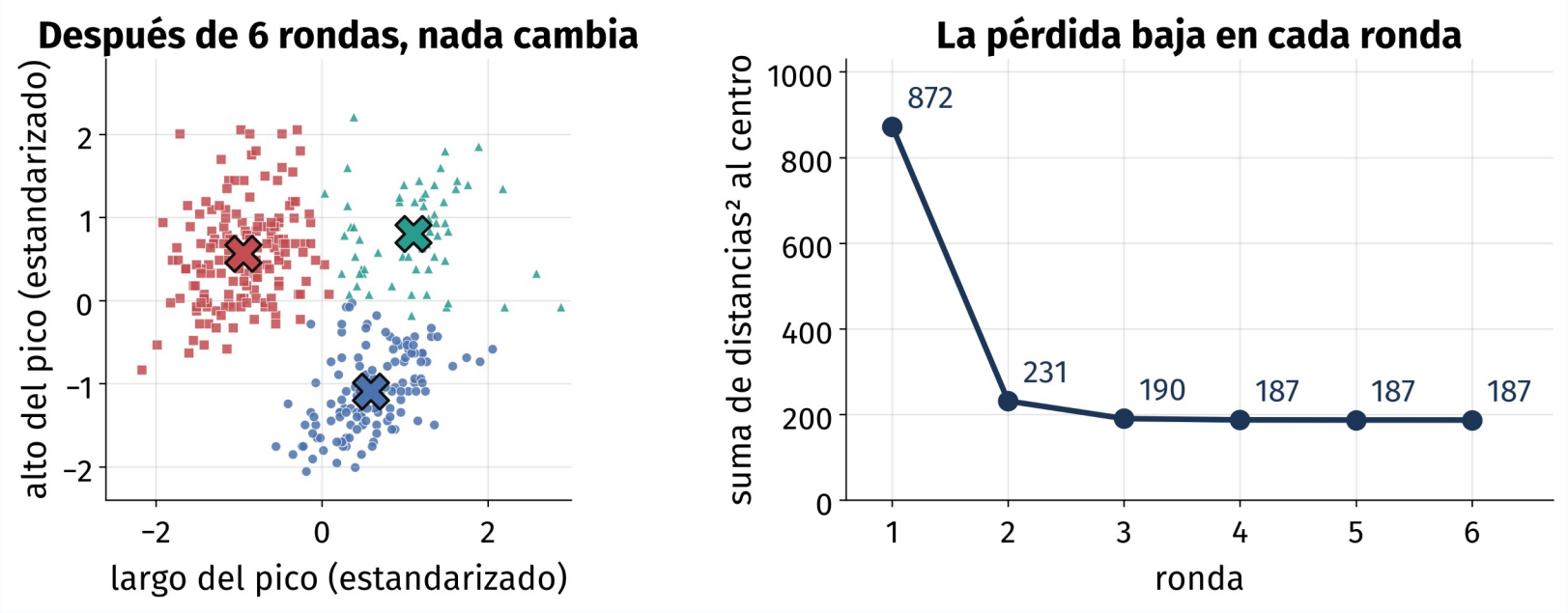
Dos medidas, estandarizadas: el largo y el alto del pico. Los tres centros parten en tres pingüinos elegidos al azar y cada pingüino va al centro más cercano. Pérdida: 872.

Ejemplo: actualización de centros y reasignación



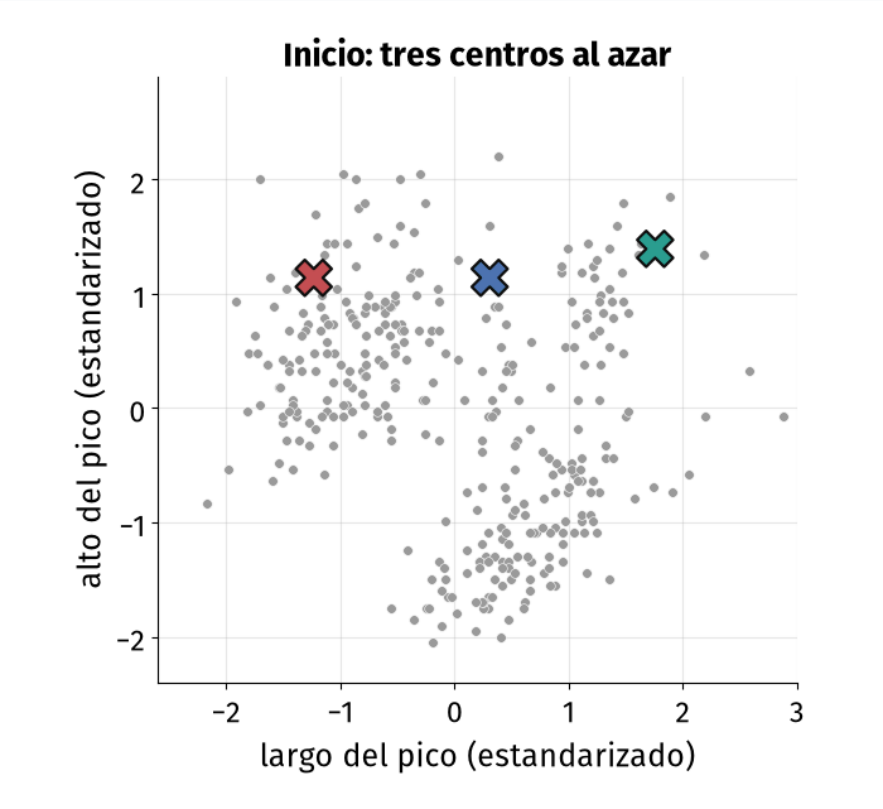
Cada centro se mueve al promedio de su grupo (las flechas parten de la posición anterior) y cada pingüino se reasigna al centro más cercano. La pérdida baja de 872 a 231.

Ejemplo: convergencia del algoritmo



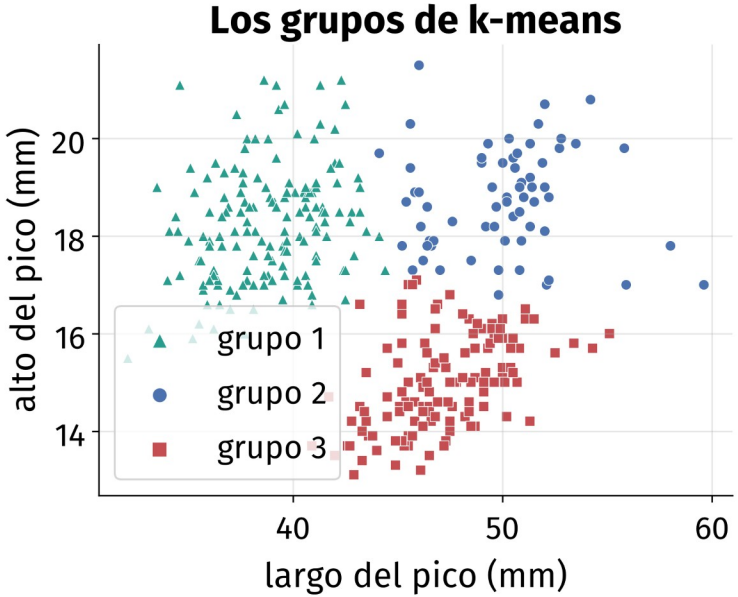
En la sexta ronda los grupos ya no cambian. La pérdida bajó en cada ronda, como asegura el algoritmo, y quedó en 187.

Animación del algoritmo de Lloyd



El mismo ejemplo, paso a paso: en cada ronda se asigna cada pingüino al centro más cercano y se mueve cada centro al promedio de su grupo, hasta que nada cambia. La pérdida baja de 872 a 187.

Comparación de los grupos con las especies



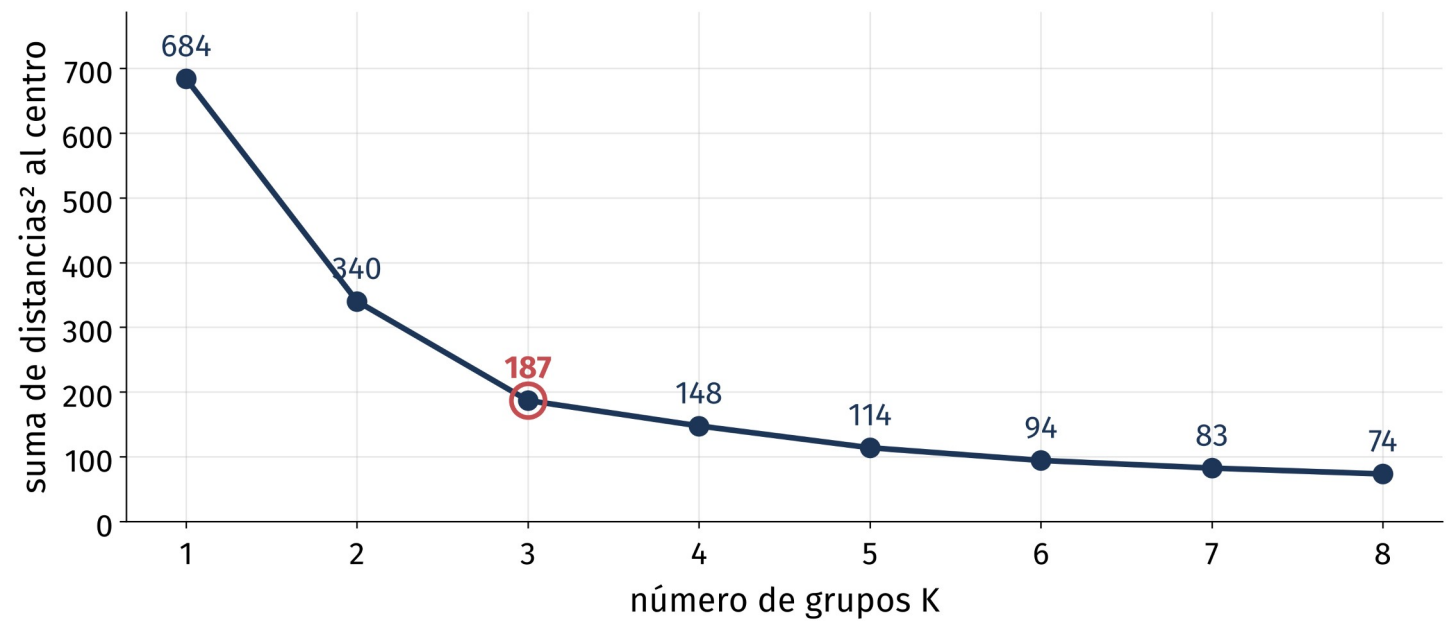
¿Coinciden con las especies?

	grupo 1	grupo 2	grupo 3
Adelia	**147**	4	0
Barbijo	5	**54**	9
Papúa	1	6	**116**

317 de 342 pingüinos (93%) quedan en el grupo de su especie, sin que k-means la viera

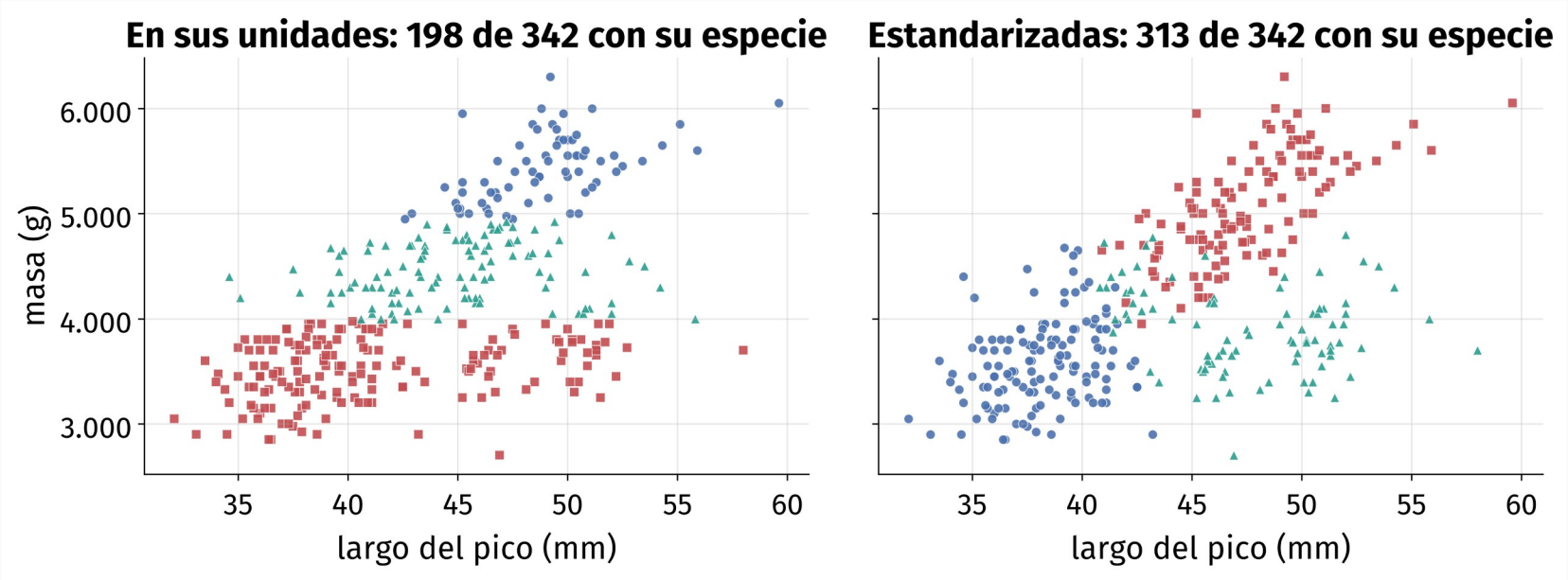
k-means no vio la especie. Al comparar después, 317 de los 342 pingüinos quedan en el grupo de su especie; los que no, son sobre todo barbijo, que se traslapan con los otros dos. Esta comprobación es posible porque aquí se conoce la especie; en general no se conoce.

Elección del número de grupos: elbow method



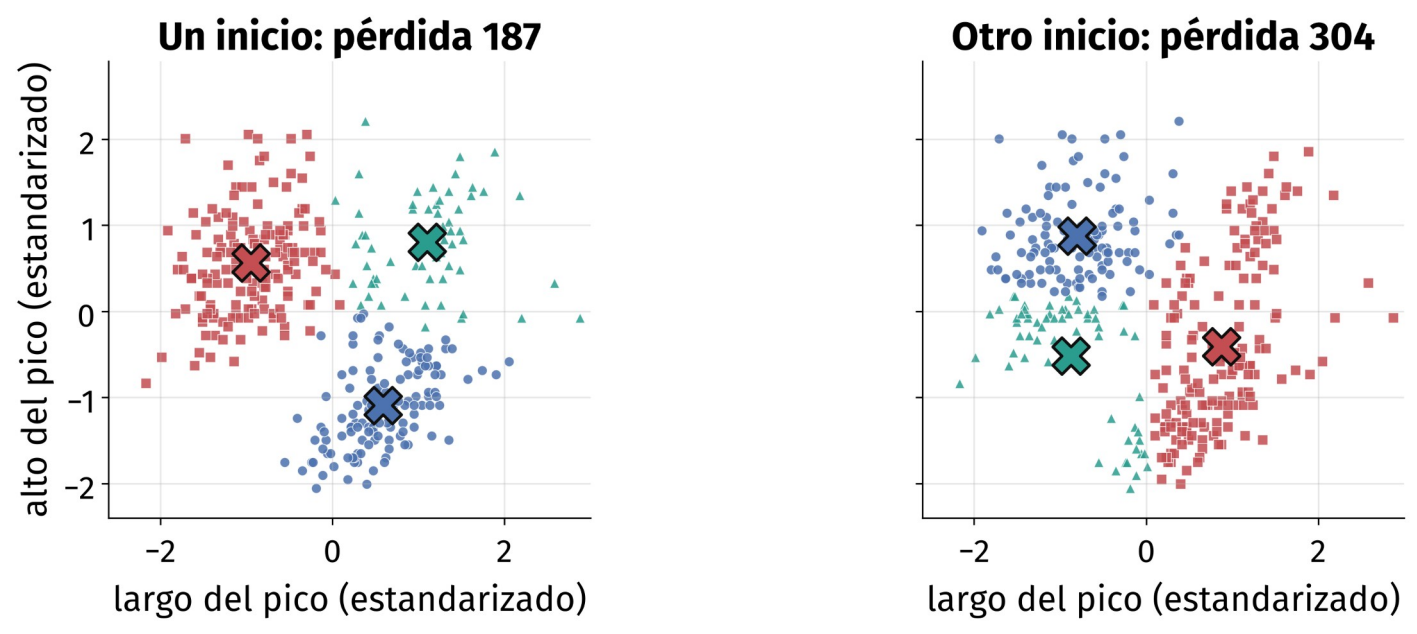
La pérdida siempre baja al agregar grupos: con un grupo por pingüino sería 0. El quiebre está en K = 3 (de 684 a 340 y a 187; después baja poco). Como en PCA, no hay una única respuesta correcta (James et al., 2013).

Efecto de la estandarización en k-means



Con las cuatro medidas en sus unidades, la masa (desviación estándar de 802 g) pesa mucho más que el pico (2 y 5,5 mm): los grupos son franjas de masa y solo 198 de 342 pingüinos quedan con su especie. Con las cuatro estandarizadas, 313.

Sensibilidad a la inicialización



El algoritmo llega a un mínimo local. De 300 inicios al azar, 2 terminan en una solución peor (pérdida 304 en vez de 187). Por eso se corre varias veces y se queda la de menor pérdida (James et al., 2013); en scikit-learn lo hace n_init.

Limitaciones de k-means

- **K lo elige quien analiza: el codo orienta, no decide.**
- k-means siempre entrega grupos, aunque los datos no tengan grupos: falta comprobar que no sean ruido (James et al., 2013).
- Interpretar los grupos requiere conocer el problema: aquí se pudo con las especies, que en general no se tienen.

Implementación de k-means en scikit-learn

```
Z = StandardScaler().fit_transform(X)          # 1. estandarizar
km = KMeans(n_clusters=3, n_init=10).fit(Z)    # 2. los grupos
km.labels_                                     # 3. el grupo de cada uno
km.cluster_centers_                            # 4. los centros
km.inertia_                                    # 5. la pérdida
```

- El ejemplo completo, con los pingüinos, está en el mismo notebook.

Próximas clases

- **Viernes 6 de noviembre: clustering jerárquico, máquinas de soporte vectorial y el avance opcional de proyecto.**
- Lunes 16 de noviembre: se entrega la Tarea 2 por Aula.
- Viernes 20 de noviembre: Control Q4 al inicio de la clase.

Referencias (1 de 2)

James, G., Witten, D., Hastie, T. y Tibshirani, R. (2013). An Introduction to Statistical Learning. Springer. Sección 2.1.4 (supervisado y no supervisado); sección 10.2 (componentes principales); sección 10.3 (k-means); sección 6.3.1 (regresión sobre componentes). PDF gratuito en statlearning.com.

Hastie, T., Tibshirani, R. y Friedman, J. (2009). The Elements of Statistical Learning (2.ª ed.). Springer. Sección 2.4 (función de pérdida).

Jordan, M. I. y Mitchell, T. M. (2015). Machine learning: Trends, perspectives, and prospects. Science, 349(6245), 255 a 260.

Referencias (2 de 2)

Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. Philosophical Magazine, 2(11), 559 a 572.

MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, 1, 281 a 297.

Lloyd, S. P. (1982). Least squares quantization in PCM. IEEE Transactions on Information Theory, 28(2), 129 a 137.

Gorman, K. B., Williams, T. D. y Fraser, W. R. (2014). Ecological sexual dimorphism and environmental variability within a community of Antarctic penguins (genus Pygoscelis). PLoS ONE, 9(3), e90081.

Datos: Hitters, de James et al. (en datos/clase07/), y los pingüinos de Palmer, del paquete palmerpenguins, licencia CC0 (en datos/clase08/). Documentación de scikit-learn: PCA y KMeans.